Задача 1. Сортировка деревом поиска

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В первых четырёх байтах входного файла задано целое число N — количество чисел в массиве A. Далее идут N четырёхбайтовых целых чисел — содержимое массива A. Размер массива лежит в диапазоне: $0 \le N \le 500\,000$.

Требуется отсортировать массив A по не убыванию, используя **дерево поиска**. Учтите, что в исходном массиве может быть много одинаковых элементов. Кроме того, элементы массива могут быть изначально выстроены в каком-то фиксированном порядке.

В выходной файл нужно вывести ровно N четырёхбайтовых целых чисел: содержимое массива A после сортировки.

Пример

input.txt
0A 00 00 00 1F 00 00 00 F2 FF FF FF 06 00 00 00 04 00 00 00 26 00 00 00 FD FF FF FF 1E 00 00 00 F6 FF FF FF 0A 00 00 00 F4 FF FF FF
output.txt
F2 FF FF FF F4 FF FF FF F6 FF FF FF FD FF FF FF 04 00 00 00 06 00 00 00 0A 00 00 00 1E 00 00 00 1F 00 00 00 26 00 00 00

Задача 2. Аллокатор

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Функции malloc и free позволяют динамически выделять блоки памяти и возвращать их назад в кучу. К сожалению, иногда эти функции работают медленнее, чем того хотелось бы. В таких случаях программисты порой реализуют собственные алгоритмы выделения памяти взамен malloc/free, которые применимы в одной конкретной задаче, зато работают при этом намного быстрее.

В этой задаче нужно реализовать специальный алгоритм для выделения блоков памяти одинакового (и маленького) размера. Алгоритм работает очень просто. Изначально выделяется большой кусок памяти (по сути, массив) размером ровно в N блоков. Когда пользователь запрашивает у аллокатора новый блок памяти, аллокатор выбирает любой незанятый блок из этого массива и возвращает его адрес пользователю. Если все блоки заняты, аллокатор должен вернуть нулевой указатель, так как заведуемая им память закончилась. Когда пользователь освобождает блок памяти, аллокатор помечает его как свободный, чтобы в будущем можно было его переиспользовать.

Кроме собственно массива блоков, аллокатор также должен хранить множество незанятых блоков, чтобы знать, какие блоки сейчас можно выдавать пользователю, а какие нет.

Для этого используется система под названием free list. Все незанятые блоки объединяются в односвязный список, причём узлами этого списка становятся сами незанятые блоки памяти. То есть в каждом незанятом блоке аллокатор хранит указатель на следующий такой незанятый блок.

Обратите внимание, что узлы односвязного списка **физически расположены внутри того самого массива**, блоки которого выдаются пользователю, а не где-то ещё снаружи!

Так получается аллокатор без накладных расходов: помимо собственно куска памяти из N блоков не нужно никакой дополнительной памяти, кроме $O(1)$ памяти где-то в головной структуре аллокатора.

В тестовой задаче нужно реализовать аллокатор для выделения блоков размером в 8 байт и записи туда вещественных значений типа double. Нужно реализовать следующие функции:

```
typedef struct MyDoubleHeap_s
{
    ??? // можно хранить здесь всякие данные
} MyDoubleHeap;
// создать новый аллокатор с массивом на slotsCount блоков
MyDoubleHeap initAllocator(int slotsCount);
// запросить блок памяти под число типа double
double *allocDouble(MyDoubleHeap *heap);
// освободить блок памяти, на который смотрит заданный указатель
void freeDouble(MyDoubleHeap *heap, double *ptr);
```

Далее нужно обработать набор операций/запросов.

Формат входных данных

В первой строке задано два целых числа: N — на сколько блоков нужно изначально создать массив (slotsCount) и Q — сколько операций нужно после этого выполнить ($2 \le N, Q \le 3 \cdot 10^5$). В остальных Q строках описаны операции.

Каждая операция начинается с целого числа t — типа операции. Если $t = 0$, то это операция выделения блока памяти. Тогда далее записано вещественное число, которое нужно сохранить в этом блоке памяти. При выполнении этой операции нужно вывести в выходной файл адрес, который вернула функция allocDouble. Этот адрес должен должен делиться на 8, чтобы double был корректно выровнен по своему размеру.

Если $t = 1$, то это операция освобождения блока памяти, и далее записано целое число k — номер операции, в которой был выделен тот блок памяти, который сейчас нужно удалить. Если $t = 2$, то нужно просто распечатать содержимое того блока памяти, который был выделен на k-ой операции, как вещественное число.

Все запросы нумеруются по порядку номерами от 0 до $Q - 1$. Для вывода в файл адреса/указателя используйте формат "%p". Все вещественные числа заданы с не более чем 5 знаками после десятичной точки, и не превышают 104 по модулю. Вещественные числа следует выводить в аналогичном виде (например, используя формат "%0.5lf").

Гарантируется, что никакой выделенный блок памяти не будет удалён дважды, и что у вас не попросят распечатать содержимое уже освобождённого блока. Гарантируется, что если запрос на выделение памяти возвращает нулевой указатель (когда все N блоков заняты), то на эту операцию не ссылаются никакие другие запросы, т.е. этот невыделенный блок не попытаются освободить или распечатать.

Формат выходных данных

Выведите результаты выполнения операций (для операций типа $t = 0$ и $t = 2$).

Пример

input.txt	output.txt
5 12 0 0.1 0 1.1 0 2.1 0 3.1 0 4.1 0 5.1 2 3 2 1 1 3 0 123.0 2 9 0 -1	001A0480 001A0488 001A0490 001A0498 001A04A0 00000000 3.100000000000000 1.100000000000000 001A0498 123.000000000000000 00000000

Пояснение к примеру

Изначально вызываем initAllocator с slotsCount = 5. Внутри он создаёт массив на 5 элементов, и объединяет их все в односвязный список (т.к. изначально все блоки незаняты).
Далее делается шесть запросов на выделение памяти. Первые пять срабатывают успешно, и в выходном файле распечатаны адреса блоков (они идут подряд с шагом в 8 байт). Для шестого запроса свободных блоков нет (ведь $N = 5$), поэтому блок не выделяется и выводится нулевой указатель.

Задача 3. Динамический поиск

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	2 секунды
Ограничение по памяти:	**разумное**

Имеется множество целых чисел X, изначально оно пустое. Нужно выполнить M заданных операций над этим множеством.

Есть три типа операций:

1. **add** v — добавить число v в множество X. Если такого числа ещё не было в множестве, надо его добавить и напечатать слово added. Если такое число уже есть в множестве, нужно напечатать слово dupe и ничего не делать.

2. **remove** v — удалить число v из множества X. Если такое число есть в множестве, нужно его удалить и напечатать слово removed. А если такого числа нет, нужно напечатать слово miss и ничего не делать.

3. **lower** v — найти минимальное число в множестве X, которое больше или равно заданному v (т.е. lower_bound). Если такое число в множестве есть, нужно напечатать в файл. А если его нет, то есть если v больше всех чисел множества X, то нужно напечатать ### (три символа решётки, ASCII 35).

Внимание: операции нужно выполнять в режиме “online”: считывать операцию из файла разрешается только после того, как все предыдущие операции уже выполнены. Задачу нужно решать, используя **сбалансированное дерево поиска**.

Формат входных данных

В первой строке содержится целое число M — количество операций ($1 \le M \le 3 \cdot 10^5$).
В остальных M строках записаны операции в порядке их выполнения. Все числа v в файлецелые и по абсолютной величине не превышают 10^9.

Формат выходных данных

Нужно вывести M строк, в каждой из которых требуется записать результат выполнения соответствующей операции.

Пример

input.txt	output.txt
16 add 7 add 3 add 5 add 5 add 10 add 7 remove 6 remove 5 add 5 remove 3 lower 2 lower 5 add 1 lower 0 lower 10 lower 15	added added added dupe added dupe miss removed added removed 5 5 added 1 10 ###

input.txt
5 "O]V| c -(SZ9mY ~‘/{8" 1950 "bo" " Q G*u4 T:Eqy’d" 1979 "9" "0" 2005 "jMsB" " w cQ" 1982 "&" " Q G*u4 T:Eqy’d" 2003 " J|hL" 7 " Q G*u4 T:Eqy’d" "6q*EDh!" "0" "s" " Q G*u4 T:Eqy’d" "CQMG::dw{" ":D h%$ W^cr" "%‘De!Si" " Q G*u4 T:Eqy’d" "]Zo" " Q G*u4 T:Eqy’d" "t" "0" "Uxb/.& "
output.txt
" Q G*u4 T:Eqy’d" 1979 "9" " Q G*u4 T:Eqy’d" "t" " Q G*u4 T:Eqy’d" 1979 "9" " Q G*u4 T:Eqy’d" "CQMG::dw{" " Q G*u4 T:Eqy’d" 1979 "9" " Q G*u4 T:Eqy’d" "]Zo" " Q G*u4 T:Eqy’d" 1979 "9" " Q G*u4 T:Eqy’d" "6q*EDh!" " Q G*u4 T:Eqy’d" 2003 " J|hL" " Q G*u4 T:Eqy’d" "t" " Q G*u4 T:Eqy’d" 2003 " J|hL" " Q G*u4 T:Eqy’d" "CQMG::dw{" " Q G*u4 T:Eqy’d" 2003 " J|hL" " Q G*u4 T:Eqy’d" "]Zo" " Q G*u4 T:Eqy’d" 2003 " J|hL" " Q G*u4 T:Eqy’d" "6q*EDh!" "0" 2005 "jMsB" "0" "Uxb/.& " "0" 2005 "jMsB" "0" "s"

Задача 4. sql join: бинарный поиск

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Данная задача является продолжением задачи «sql join». Прочитайте условие оригинальной задачи перед тем, как читать дальше!

В данной задаче необходимо использовать сортировку и бинарный поиск для ускорения соединения. Отсортируйте все записи одной таблицы по имени актёра. Далее переберите все записи другой таблицы: для каждой записи можно бинарным поиском найти первую запись с тем же именем, после чего легко обнаружить их все.

Входные/выходные данные в этой задаче такие же, как в задаче «sql join». Ограничения такие же ($1 \le N, M \le 10^5$), за одним важным исключением:

В этой задаче не гарантируется, что $N \cdot M \le 10^5$. Вместо этого гарантируется, что после соединения количество записей R не превышает 10^5.

Задача 5. sql join: слияние

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Данная задача является продолжением задачи «sql join». Прочитайте условие оригинальной задачи перед тем, как читать дальше!
В данной задаче необходимо использовать сортировку и слияние для ускорения соединения. Отсортируйте обе таблицы по имени актёра. Теперь вы можете двигать два указателя, каждый по своей таблице точно так же, как в алгоритме слияния. Если по указателям имена совпадают, надо найти полностью совпадающие группы и добавить попарно их конкатенации в ответ, а если не совпадают, то надо указатель на запись с меньшим именем сдвинуть вправо.
Входные/выходные данные в этой задаче такие же, как в задаче «sql join». Ограничения такие же ($1 \le N, M \le 10^5$), за одним важным исключением:
В этой задаче не гарантируется, что $N \cdot M \le 10^5$. Вместо этого гарантируется, что после соединения количество записей R не превышает 10^5.

Задача 6. Максимум в окне

Источник:	основная
Имя входного файла:	stdin
Имя выходного файла:	stdout
Ограничение по времени:	3 секунды
Ограничение по памяти:	разумное

Дан массив A, в котором записано N целых чисел. По этому массиву перемещается окно.

Окно — это подотрезок в массиве, начинающийся с L-ого элемента массива, и заканчивающийся на (R-1)-ом элементе (всего в окне $R - L$ элементов). При этом счётчики L и R постоянно изменяются. Нужно после каждого изменения счётчиков найти и вывести максимальное число в окне.

Изначально оба счётчика равны 0. Далее нужно выполнить $2N - 1$ операций, каждая операция имеет один из двух типов:

- L — увеличить счётчик L на 1, тем самым сдвинув начало окна.
- R — увеличить счётчик R на 1, тем самым сдвинув конец окна.

После выполнения каждой операции нужно вывести максимум среди чисел в окне, то есть максимум среди элементов массива, индекс которых попадает в диапазон $[L, R)$.

Формат входного файла

В первой строке содержится целое число N — количество элементов в массиве ($1 \le N \le 2 \cdot 10^5$). Во второй строке записано N целых чисел через пробел — содержимое массива. Все числа по абсолютной величине не превышают 10^9. В третьей и последней строке записано подряд ($2N - 1$) символов — команды, которые требуется выполнить.

Гарантируется, что:

1. первая команда имеет тип R,

2. после выполнения каждой команды верно R

Формат выходного файла

Нужно вывести $2N - 1$ строк, в каждой из которых требуется записать максимум в текущем окне. Максимум нужно выводить после обработки каждой команды.

Пример

input.txt	output.txt
14 1 8 3 2 5 2 7 3 7 4 9 1 3 2 RRRRLLRLRRLLRRLLLRRRRLRLLLL	1 8 8 8 8 3 5 5 5 7 7 7 7 7 7 7 7 7 9 9 9 9 9 9 3 3 2

Задача 7. sql join: дерево поиска

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	2 секунды
Ограничение по памяти:	разумное

Данная задача является продолжением задачи «sql join». Прочитайте условие оригинальной задачи перед тем, как читать дальше!

В данной задаче необходимо использовать дерево поиска для ускорения соединения. Занесите все записи одной таблицы в бинарное дерево поиска (при этом нужно решить, что делать с равными ключами). Далее переберите все записи второй таблицы: для каждой из них можно найти в дереве поиска список всех записей с совпадающим именем.

Входные/выходные данные в этой задаче такие же, как в задаче «sql join». Ограничения такие же ($1 \le N, M \le 10^5$), за одним важным исключением:

В этой задаче не гарантируется, что $N \cdot M \le 10^5$. Вместо этого гарантируется, что после соединения количество записей R не превышает 10^5.

Задача 8. Scapegoat

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Нужно реализовать структуру данных Scapegoat Tree — самобалансирующееся бинарное дерево поиска с амортизированными гарантиями O(log n) на операции вставки и удаления и точными O(log n) для поиска.

Формат входного файла

Первая строка — целое число Q ($1 \le Q \le 2 \cdot 10^5$) — количество операций. Далее Q строк, каждая из которых описывает одну операцию:

- + x – вставить число x
- ? x – проверить, есть ли число x в множестве
- p – печать дерева

Гарантируется, что $|x| \le 10^9$.

Формат выходного файла

Для каждой операции ? x выведите в отдельной строке:

- Found — если x присутствует в множестве
- Not found — если x отсутствует

Пример

input.txt	output.txt
9 + 5 + 3 + 8 ? 5 + 5 ? 3 + 1000000000 ? 1000000000 p	Found Found Found 3 5 8 1000000000

Задача 9. Cartesian на языке Си

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Нужно реализовать структуру данных Cartesian Tree — бинарное дерево поиска с функциями merge, split, и основанными на них функциями lookup, insert, remove, работающие в среднем за O(log n). Скорость работы O(n). Количество деревьев – не более 1000.

Формат входного файла

Первая строка — целое число Q ($1 \le Q \le 2 \cdot 10^5$) — количество операций. Далее Q строк, каждая из которых описывает одну операцию:

- insert x – вставить число x
- lookup x – проверить, есть ли число x в множестве
- remove x – удалить число из x

Гарантируется, что $|x| \le 10^9$.

Формат выходного файла

Для каждой операции lookup x выведите в отдельной строке:

- Found — если x присутствует в множестве
- Not found — если x отсутствует

Пример

input.txt	output.txt
9 insert 5 insert 3 insert 8 lookup 5 insert 5 remove 3 lookup 3 insert 1000000000 lookup 1000000000	Found Not found Found

Задача 10. Дерево ван Эмде Боаса

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дерево ван Эмде Боаса изобрел в 1975 году голландский ученый-компьютерщик Петер ван Эмде Боас. Эта структура очень полезная для целых чисел [0, U), где $U = 2^k$. Для таких чисел скорость поиска, вставки, удаления составляет $O(loglogU)$, однако она имеет довольно большую константу памяти $O(U)$ и бесполезна для разреженных наборов чисел. Тем не менее для создания приоритетных очередей, сортировка последовательности $O(nloglogU)$, нахождения наибольшей возрастающей подпоследовательности $O(nloglogU)$, алгоритма Дейкстры $O(EloglogU)$ и т.п. это дерево оказывается полезным. Ваша задача реализовать дерево ван Эмде Боаса.

Формат входного файла

В первой строке дано максимальное число, встречающееся в дереве. В последующих строках заданы операции insert - вставка, find - поиск, print - печать.

Пример

input.txt	output.txt
10 insert 5 print insert 10 insert 3 find 5 find 15 find 17 print	5 Found Not found Not found 3 5 10

Задача 11. Подмассивы

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В 1 строке задан массив целых int32 чисел длиной L ($1 \le L \le 10^6$).

В последующих Q ($1 \le Q \le 10^6$)строках заданы левая и правая границы массива. Для каждого подмассива вывести целое число – минимальный элемент подмассива.

Формат входного файла

В 1 строке размер массива и количество запросов.
Через пробел записаны целые числа подмассива.
Далее Q строк, состоящих из пар чисел l r.

Формат выходного файла

Для пары чисел l r выведите в отдельной строке минимальный элемент подстроки.

Пример

input.txt	output.txt
6 2 1 5 4 2 3 6 1 5 4 5	2 3

Задача 12. Suffix tree

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Нужно реализовать структуру данных Suffix tree для поиска подстроки в строке за O(P).

Формат входного файла

В 1 строке дана строка состоящая из печатаемых символов
В последующих строках заданы подстроки. Нужно найти индексы вхождений.
В конце распечатайте полученное дерево.
Если элементов в дереве больше 100, печать дерева не нужна.

Комментарий

При сдаче можно предложить подумать о реализации обобщенного суффиксного дерева.

Пример

input.txt	output.txt
student_otchislen_za_neznanie student _ o za nez	1 8 9 19 22 root +s |+tudent_otchislen_za_neznanie$ |+len_za_neznanie$ +t |+udent_otchislen_za_neznanie$ |+_otchislen_za_neznanie$ |+chislen_za_neznanie$ +udent_otchislen_za_neznanie$ +dent_otchislen_za_neznanie$ +e |+n ||+t_otchislen_za_neznanie$ ||+_za_neznanie$ |+znanie$ |+$ +n |+t_otchislen_za_neznanie$ |+_za_neznanie$ |+eznanie$ |+anie$ |+ie$ +_ |+otchislen_za_neznanie$ |+za_neznanie$ |+neznanie$ +otchislen_za_neznanie$ +chislen_za_neznanie$ +hislen_za_neznanie$ +i |+slen_za_neznanie$ |+e$ +len_za_neznanie$ +z |+a_neznanie$ |+nanie$ +a |+_neznanie$ |+nie$ +$

Задача 13. Фибоначчиевы пирамиды

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Нужно реализовать структуру данных «Фибоначчиева пирамида» — набор корневых деревьев, упорядоченных в соответствии со свойством неубывающей пирамиды, т.е. каждое дерево подчиняется этому свойству: ключ узла не меньше ключа его родителя.

Формат входного файла

- create – создает новую пирамиду фибоначчи (не более 10000 пирамид)
- insert – вставляет новый элемент в последнюю созданную пирамиду
- find – ищет минимальный элемент в 1 созданной пирамиде (если его нет пустая строка)
- merge – слияние всех пирамид в первую
- extract - извлекает минимальный элемент в 1 созданной пирамиде (если его нет пустая строка)
- empty – 0 – если не пустая 1 созданная пирамида, 1 – если пустая.
- print – печать первой пирамиды (или пустая строка)
- quit – выход из программы

Пример

input.txt	output.txt
create insert 10 find create insert 3 create insert 7 insert 9 find merge find extract extract empty extract empty print quit	10 10 3 3 7 0 9 0 10